CNN-FPGA 协同 GKP 解码工程化实验方案（可落地实施版）

0. 文档目标

本文档用于指导下一阶段 CNN-FPGA 协同 GKP 解码实验的工程化实施，目标是：

1. 在保留原《CNN_FPGA_GKP_实验设计》研究目标的前提下，补齐可执行的项目结构与模块边界。
2. 将“物理仿真 -> 硬件行为仿真 -> 硬件在环(HIL)”串成同一套可验证流程。
3. 明确每个阶段的输入输出、接口协议、延迟预算、验证指标和验收门槛。

该方案强调工程可行性，优先采用可逐步替换、可回退、可量化验收的实现路径。

1. 总体技术路线

1.1 双回路控制保持不变

- 快回路（FPGA，周期级）
  - 输入：每轮 syndrome（`s_q, s_p`）
  - 计算：线性解码 `Δ = K @ s + b`
  - 输出：校正位移指令
  - 并行任务：32x32 直方图在线累积
- 慢回路（ARM/CNN，窗口级）
  - 输入：快回路累积的直方图窗口
  - 计算：CNN 预测噪声参数（`σ, μ_q, μ_p, θ`）
  - 输出：更新解码器参数（`K, b`）

1.2 三层验证闭环

1. 物理层仿真：基于 `physics` 模块生成 syndrome 与逻辑错误统计。
2. 硬件行为仿真：加入固定点量化、寄存器/总线延迟、DMA 读写延迟、参数切换保护。
3. 硬件在环：FPGA 快回路 + ARM 慢回路真实运行，软件负责驱动和记录。

2. 建议项目结构（新增模块）

```
DriftAdaptiveQEC/
├── physics/                              # 现有物理仿真
│   ├── gkp_state.py
│   ├── noise_channels.py
│   ├── syndrome_measurement.py
│   ├── error_correction.py
│   └── logical_tracking.py
│
├── cnn_fpga/                             # 新增：工程主模块
│   ├── __init__.py
│   ├── config/
│   │   ├── experiment_static.yaml
│   │   ├── experiment_drift.yaml
│   │   ├── hardware_emulation.yaml
│   │   └── hardware_hil.yaml
│   ├── data/
│   │   ├── schema.py                     # 数据结构定义（syndrome/hist/labels）
│   │   ├── dataset_builder.py            # 数据集生成
│   │   ├── histogram.py                  # 直方图映射和归一化
│   │   └── split.py                      # train/val/test 划分
│   ├── model/
│   │   ├── tiny_cnn.py                   # CNN 模型定义
│   │   ├── train.py                      # 训练入口
│   │   ├── evaluate.py                   # 回归评估
│   │   ├── quantize.py                   # QAT/PTQ + int8 导出
│   │   └── export.py                     # ONNX/TFLite 导出
│   ├── decoder/
│   │   ├── param_mapper.py               # (σ,μ,θ) -> (K,b)
│   │   ├── linear_runtime.py             # 快回路等价软件实现
│   │   ├── mwpm_stub.py                  # MWPM 接口占位（后续实现）
│   │   └── ekf_baseline.py               # EKF 基线
│   ├── runtime/
│   │   ├── fast_loop_emulator.py         # 带周期约束的快回路仿真
│   │   ├── slow_loop_runtime.py          # CNN 推理与参数更新
│   │   ├── scheduler.py                  # 双回路调度
│   │   ├── param_bank.py                 # 双缓冲参数切换
│   │   └── latency_injector.py           # DMA/AXI/推理延迟注入
│   ├── hwio/
│   │   ├── axi_map.py                    # 寄存器映射定义
│   │   ├── dma_client.py                 # DMA 读写接口
```

```
│   │   ├── fpga_driver.py                # HIL 驱动封装
│   │   └── mock_fpga.py                  # 无板卡时模拟后端
│   ├── benchmark/
│   │   ├── run_static_suite.py
│   │   ├── run_drift_suite.py
│   │   ├── run_hil_suite.py
│   │   └── summarize.py
│   └── report/
│       ├── metrics.py
│       ├── plots.py
│       └── export_markdown.py
│
├── fpga/                                 # 新增：FPGA 实现目录
│   ├── rtl/
│   │   ├── linear_decoder.v
│   │   ├── histogram_accum.v
│   │   ├── param_bank.v
│   │   └── axi_lite_regs.v
│   ├── hls/
│   │   └── cnn_preprocess_hls.cpp        # 可选
│   ├── sim/
│   │   ├── tb_linear_decoder.sv
│   │   └── tb_histogram.sv
│   ├── constraints/
│   │   └── timing.xdc
│   └── vivado/
│       └── project.tcl
│
├── benchmark/
│   └── compare_full_vs_simplified_ler.py  # 已有最小对比脚本
│
└── docs/
    ├── CNN_FPGA_GKP_实验设计.md
    └── CNN_FPGA_GKP_工程化实验方案.md
```

3. 模块接口与数据契约

3.1 syndrome 数据格式

- `float32[2]` ： `[s_q, s_p]`

- 快回路内部固定点： `Q4.20` （建议）
- 量程建议： `[-lambda/2, +lambda/2]` ，超限做饱和而非截断回绕

3.2 直方图格式

- 网格： `32 x 32`
- bin 计数： `uint16` 或 `uint32` （根据窗口大小选择）
- 窗口长度：建议 `W=1024` 或 `W=2048` 周期
- 归一化策略：
  i. `H = H / sum(H)`
  ii. 可选 `log1p(alpha*H)` 以增强弱信号分辨率

3.3 参数更新格式

- CNN 输出： `[σ, μ_q, μ_p, θ]`
- 映射输出： `K(2x2) + b(2,)` ，共 6 个参数
- FPGA 寄存器写入固定点： `Q4.20`
- 参数切换策略：双缓冲
  - `bank_A` 运行中
  - `bank_B` 写入中
  - `commit_epoch` 到达后在周期边界原子切换

3.4 AXI-Lite 地址建议

- `0x00` `CTRL` ：bit0 start, bit1 reset_hist, bit2 commit_bank
- `0x04` `STATUS` ：bit0 ready, bit1 hist_ready, bit2 commit_ack
- `0x10~0x24` ： `K11,K12,K21,K22,b1,b2`
- `0x30` ： `ACTIVE_BANK`
- `0x34` ： `EPOCH_ID`

3.5 DMA 缓冲建议

- `histogram_buffer` : 1024 bins x 4B = 4096B
- 双缓冲 DMA： `buf0/buf1` 轮转
- ARM 端读取目标：每 10~50 ms 一次

3.6 参数映射与稳定化规则（必须固定，避免训练-部署语义漂移）

给定 CNN 预测输出 `(sigma_hat, mu_q_hat, mu_p_hat, theta_hat)` ，建议采用以下确定性映射：

1. 增益估计（Wiener 型）
   `g_raw = var_signal / (var_signal + sigma_hat^2 + sigma_meas^2 + delta_eff^2)`
   `g = clip(g_raw, g_min, g_max)` ，推荐 `g_min=0.2` ，`g_max=1.2`
2. 旋转矩阵
   `R(theta_hat) = [[cos(theta), -sin(theta)], [sin(theta), cos(theta)]]` ，并限制 `theta_hat` 在 `[-20°, +20°]`
3. 解码参数
   `K_target = g * R(theta_hat)`
   `b_target = -alpha * [mu_q_hat, mu_p_hat]` ，推荐 `alpha in [0.5, 1.0]`
4. 参数平滑（防止抖动）
   `K_next = (1-beta) * K_prev + beta * K_target`
   `b_next = (1-beta) * b_prev + beta * b_target` ，推荐 `beta=0.1~0.3`

说明：参数映射一旦在实验主线确定，不得在不同基线间隐式修改，否则会破坏公平对比。

3.7 直方图边界策略（必须显式实现）

- `s_q/s_p` 超出映射范围时，不允许静默丢弃。
- 推荐策略：先 clip 到 `[s_min, s_max]` 再入 bin，并累计 `overflow_counter` 作为诊断指标。
- 每次慢回路推理时输出 `overflow_ratio = overflow_counter / W` ，若超过阈值（如 5%）触发告警。

4. 时序预算与可行性约束

4.1 目标周期

- QEC 快回路周期目标： `T_fast = 5 us` （可根据设备调到 1~10 us）
- 慢回路更新周期： `T_slow = 10~100 ms`

4.2 快回路预算（每周期）

- syndrome 输入与寄存： `<= 0.5 us`
- 线性解码（固定点矩阵运算）： `<= 0.5 us`
- 校正输出准备： `<= 0.5 us`
- 直方图累积：并行，不占关键路径

总计关键路径目标： `<= 1.5 us` ，保留裕量给 ADC/控制链路。

4.3 慢回路预算（每次更新）

- DMA 读出： `5~20 us`
- 预处理： `20~100 us`
- CNN int8 推理： `0.1~2 ms` （取决于部署目标）
- 参数映射与写回： `10~100 us`

总计目标： `< 5 ms` （保守可接受 `< 20 ms` ）。

4.4 窗口长度与更新周期耦合约束

为避免“训练设定”和“在线推理节奏”不一致，必须满足：

- `T_window = W * T_fast`
- `T_slow_update >= T_window` 时，采用“非重叠窗口”更新；
- `T_slow_update < T_window` 时，采用“滑动窗口”更新，并固定 stride（建议 `stride = W/4` ）。

建议默认：

- `T_fast = 5 us`
- `W = 2048` （ `T_window = 10.24 ms` ）
- `T_slow_update = 20 ms` （非重叠 + 一次缓冲冗余）

该配置能兼顾统计稳定性与慢回路实时性，避免过于频繁更新导致参数抖动。

5. 实验阶段规划

5.1 阶段 P0：仿真基线确认（已可开始）

目标：确认 `full_qec` 与 `simplified` 差异，建立后续评价基线。

- 使用脚本： `benchmark/compare_full_vs_simplified_ler.py`
- 输出： `LER 曲线 + summary.json`
- 验收：同参数下 `final_gap_mean > 0` 且跨重复稳定

5.2 阶段 P1：CNN 数据与训练闭环

目标：完成从 syndrome 直方图到 `(σ, μ_q, μ_p, θ)` 的回归模型。

步骤：

1. 新增数据生成器 `cnn_fpga/data/dataset_builder.py`
2. 生成静态+漂移混合数据集
3. 训练 Tiny-CNN（浮点）
4. 评估回归指标（MSE/MAE/R^2）
5. 导出 int8 模型并做精度回归

验收阈值建议：

- $R^2(\sigma) > 0.9$
- $R^2(\mu_q, \mu_p) > 0.85$
- $R^2(\theta) > 0.8$
- int8 精度退化 $< 10\%$

5.3 阶段 P2：硬件行为仿真（无板卡）

目标：在软件里尽量逼近硬件行为。

需要引入：

- 固定点量化误差（Q4.20）
- 寄存器写入延迟/抖动
- DMA 传输延迟/偶发阻塞
- 参数切换原子性检查

关键测试：

- 参数切换点不导致解码突变（glitch-free）
- 慢回路掉帧时快回路稳定运行
- 极端噪声下无数值溢出

5.4 阶段 P3：FPGA 在环实验（HIL）

目标：真实板卡上验证端到端链路。

步骤：

1. 部署 FPGA 快回路 IP（线性解码 + 直方图 + 参数bank）
2. ARM 运行 TFLite 推理服务
3. Linux 用户态驱动完成 DMA + AXI 写回
4. 同步记录周期级日志和系统级指标

验收阈值建议：

- 参数更新成功率 `> 99.9%`
- 快回路周期违约率 `< 10^-6`
- HIL 相对行为仿真 LER 偏差 `< 15%`

5.5 阶段 P4：对比实验与报告

输出：

- 静态噪声对比
- 漂移噪声追踪
- 非高斯鲁棒性
- 资源、延迟、功耗综合报告

5.6 分阶段硬门槛（Go/No-Go）

阶段	必达条件	未达处理
P0	`final_gap_mean > 0` 且重复实验标准差可控	先修正物理建模与统计代码，不进入 P1
P1	参数回归指标达阈值，int8 退化 < 10%	扩展数据集并重训，不进入 P2
P2	周期仿真无越界/无切换毛刺，延迟预算达标	修复运行时与参数bank，不进入 P3
P3	HIL 稳定运行 24h，无严重通信错误	驱动和RTL整改后复测
P4	主结论在 2 组以上噪声场景复现	增加重复实验与敏感性分析

6. 实验矩阵（建议）

6.1 静态场景

- `sigma_error`：0.10 ~ 0.50（步长 0.05）
- `theta`： `0°, 5°, 10°, 15°`
- `delta`： `0.2, 0.3, 0.4`
- 每点重复： `N=20`
- 每次轮数： `T=10,000`

6.2 漂移场景

- 线性漂移： $\sigma(t)=\sigma_0+\alpha t$ ， α 分三档
- 阶跃漂移： $\Delta\sigma$ 三档， t_{jump} 三档
- 周期漂移： A,f 组合网格
- 随机游走： σ_d 三档

6.3 非高斯与相关场景

- t 分布（自由度 $\nu=3,5,8$ ）
- 双峰混合高斯（模拟 TLS 跳变）
- 协方差旋转噪声（ $\rho=0.2,0.5,0.8$ ）

7. 评价指标与统计方法

7.1 主指标

- LER
- X/Z 分量错误率
- 参数回归 MSE/MAE/R^2
- 更新延迟、推理延迟、端到端延迟
- 资源利用率（DSP/BRAM/LUT/FF）
- 功耗与性能功耗比

7.2 统计方法

- 以“独立 run 的汇总统计”作为 t 检验样本
- 对时序曲线使用 block bootstrap 估计置信区间
- 同时报告效应量（Cohen's d）

7.3 基线集合（统一对比口径）

每个实验场景建议至少比较以下 4 条曲线：

1. 固定参数线性解码（Static Linear）
2. 窗口方差估计更新（Window Variance）
3. EKF 参数追踪（EKF）

4. CNN-FPGA 协同（本文方案）

MWPM 可作为扩展基线，但不应阻塞主实验流水线。

8. 关键工程细节（保证可实现）

8.1 固定点策略

- syndrome 输入： `Q4.20`
- K,b 参数： `Q4.20`
- 乘加结果：内部 `Q8.24` 累加后回落到 `Q4.20`
- 饱和处理：越界直接钳位到最大可表示值

8.2 参数切换保护

- 双 bank + epoch id
- 仅在周期边界切换
- 切换后读取回显寄存器确认

8.3 异常降级策略

- CNN 推理失败：保持上一版本参数
- DMA 超时：重试 1 次，失败则跳过本周期慢回路更新
- 参数非法（NaN/Inf/越界）：丢弃并报警

8.4 日志与追踪

至少记录：

- `timestamp`
- `epoch_id`
- `K,b`
- `predicted σ,μ,θ`
- 当前窗口 `LER`
- 慢回路耗时

8.5 可重复性与追溯字段（必须入日志）

每次 run 至少记录：

- `git_commit`
- `config_hash`
- `seed_data / seed_train / seed_runtime`
- `model_version` （含量化版本）
- `fpga_bitstream_version` （HIL 阶段）

缺失以上字段的结果不得进入最终对比统计。

8.6 仿真-硬件一致性校准（Sim2Real）

为避免离线结果与 HIL 偏差过大，建议执行两步校准：

1. 延迟校准：测量 DMA、AXI、推理真实延迟分布，回写到 `latency_injector` 。
2. 数值校准：对比浮点与固定点 `K,b` 下的单步 correction 误差分布，更新量化 scale。

校准后再运行 P3/P4 正式实验，降低“仿真乐观偏差”风险。

9. 与现有 physics 库的集成策略

9.1 复用点

- 噪声与测量模型： `physics/noise_channels.py` ， `physics/syndrome_measurement.py`
- 线性解码参数映射： `physics/error_correction.py`
- 逻辑错误评估： `physics/logical_tracking.py`

9.2 扩展点

- 新增 `cnn_fpga/decoder/mwpm_stub.py` 与统一解码接口
- 新增 `cnn_fpga/runtime/` 模块模拟硬件调度行为
- 新增 `fpga/` RTL/HLS 工程目录

10. 里程碑与时间建议

- 第 1 周：P0 收敛 + P1 数据流水线
- 第 2 周：P1 模型训练 + int8 导出
- 第 3 周：P2 行为仿真与调度器联调
- 第 4 周：P3 HIL 接入与端到端打通
- 第 5 周：P4 大规模实验与报告产出

11. 交付物清单

1. 可运行代码目录 `cnn_fpga/` 与 `fpga/`
2. 配置文件（静态、漂移、HIL）
3. 训练好的 float/int8 模型与版本记录
4. benchmark 输出（CSV/JSON/图）
5. HIL 实验日志与资源时序报告
6. 最终技术报告（性能、风险、限制、下一步）

12. 最小执行命令模板（后续实现后）

```
# 1）生成训练数据
python -m cnn_fpga.data.dataset_builder --config cnn_fpga/config/experiment_static.yaml

# 2）训练与评估
python -m cnn_fpga.model.train --config cnn_fpga/config/experiment_static.yaml
python -m cnn_fpga.model.evaluate --config cnn_fpga/config/experiment_static.yaml

# 3）量化导出
python -m cnn_fpga.model.quantize --config cnn_fpga/config/hardware_emulation.yaml

# 4）硬件行为仿真
python -m cnn_fpga.benchmark.run_drift_suite --config cnn_fpga/config/hardware_emulatio

# 5）HIL 测试
python -m cnn_fpga.benchmark.run_hil_suite --config cnn_fpga/config/hardware_hil.yaml
```

建议在每条命令补充显式随机种子（如 `--seed 1234` ）和输出目录（如 `--output runs/2026-...` ），保证结果可追溯。

13. 风险与应对

- 风险1：CNN 回归精度不足
  - 应对：增加混合噪声数据覆盖；采用多头损失函数；引入轻量 Transformer 作为备选
- 风险2：量化后精度明显下降
  - 应对：QAT 替代 PTQ；对输入做离线标定；分层敏感度分析
- 风险3：HIL 延迟超预算
  - 应对：减小窗口、降低更新频率、前移预处理到 FPGA/HLS
- 风险4：参数切换造成短时误码尖峰
  - 应对：双bank + 周期边界切换 + 切换后 1 个窗口观测保护

14. 与原实验文档的关系

- 原文档继续承担“研究问题与整体对比框架”角色。
- 本文档承担“工程实施与执行细节”角色。
- 两者共同用于后续论文与系统实现：
  - 原文档：为什么做
  - 本文档：怎么做、做到什么程度算通过